```
# import the file & print it out
with open("C:/Users/icwin/OneDrive - Indiana University/[B-573] Prog. Sci-Informatics/Pythc
    data = file.read()
# Remove the substring ">chr1_GL383518v1_alt" from the sequences
dna_data = data[20:]
dna_data
```

```
'\nctataatcccagcttgttgggaggccaaggcaggaggatcacttgaagcc\ncaggagtttgagacgagcctaagcaacatagca
```

Part 1

```
# print the 10th letter of the sequence
print(dna_data[9])
# print the 758th letter of the sequence
print(dna_data[757])
```

```
c
c
```

Part 2

```
def reverse_func(sequence):
    # Reverse the sequence
    reverse_dna = sequence[::-1]

    # Change the base pairs
    complement = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C',
                  'a': 'T', 't': 'A', 'c': 'G', 'g': 'C'}
    reverse_complement_dna = ''.join(complement.get(base, base) for base in reverse_dna)
    # print out the 79th base pair and the base pairs 500-800
    print(
        f"The 79th base pair is: {reverse_complement_dna[78]}\n"
        f"Base pairs 500-800 are: {reverse_complement_dna[499:799]}"
    )

print(reverse_func(dna_data))
```

```
The 79th base pair is: G
Base pairs 500-800 are:
GACTTCAGGTGATCCACCCGCCTTGGCCTACCATAGTGCTGGGATTATAG
GCGTGAGCCACTGTGCCTGGCCAGAATTAGACGTAGATTTTAATCATGAC
TTTCCTATTTGTTGTGTTTCTTGGACAGGTTTATACTAAATAAGAAATTA
TGTAGTATACTAAATAATACTATTTTTTTTGAAATGGAGTGTCTCTCTGT
CACCCAGGCTGGAGTGCAGTGGCGGGATCTCGGCTTACCGCAACCTCCGC
```

```
CTCCCAGGTTCAAGCAATTCTCCTGTCACAGCCTCCCAAATAGC
None
```

Part 3

```
def count_base(sequence):
    # initialize the dictionary
    dna_dict = {'A': 0, 'T': 0, 'C': 0, 'G': 0}
    # count the number of each base pair
    for i in sequence:
        if i == 'A':
            dna_dict['A'] += 1
        elif i == 'T':
            dna_dict['T'] += 1
        elif i == 'C':
            dna_dict['C'] += 1
        elif i == 'G':
            dna_dict['G'] += 1
    return dna_dict

print(count_base(dna_data))
```

```
{'A': 20805, 'T': 21542, 'C': 18949, 'G': 18845}
```

Part 4

```
# part a
def count_nucleotides(sequence):
    # count the number of each base pair in the first 1000 base pairs
    first_1000_bp = sequence[:1000]
    # initialize the list
    counts = [first_1000_bp.count(nucleotide) for nucleotide in 'ACGT']
    return counts
print(count_nucleotides(dna_data))

# part b & c
def count_nucleotides_per_kb(sequence):
    # count the number of each base pair in each 1000 base pairs
    kb_counts = []
    # loop through the sequence
    for i in range(0, len(sequence), 1000):
        # get the 1000 base pairs
        kb_segment = sequence[i:i+1000]
        # initialize the list
        counts = [kb_segment.count(nucleotide) for nucleotide in 'ACGT']
        # append the list to the list
        kb_counts.append(counts)
```

```
        kb_counts.append(counts)
    return kb_counts
print(count_nucleotides_per_kb(dna_data))

# part d
def sum_nucleotide_counts(kb_counts):
    # sum the counts of each base pair
    summed_counts = [sum(counts) for counts in kb_counts]
    return summed_counts

# print the summed counts
kb_counts = count_nucleotides_per_kb(dna_data)
summed_counts = sum_nucleotide_counts(kb_counts)
print("Summed counts:", summed_counts)
```

```
[22, 5, 4, 24]
[[22, 5, 4, 24], [46, 25, 20, 49], [2, 3, 3, 1], [61, 48, 27, 57], [73, 72, 51, 98], [31
Summed counts: [55, 140, 9, 193, 294, 107, 35, 2, 27, 18, 43, 4, 82, 480, 315, 17, 16, 2
```

Question 1

The expected sum for each list is a 1000.

Question 2

There were a few lists that had less than the expected output.

Question 3

There is a slight difference in the expected and actual results because the sequences is not a perfect palindrome.